QA Acceptance Testing — Progress Summary

Owner: Nick (QA lead) | **Last updated:** 2026-07-05

What this is

An automated acceptance-test suite at `backend/src/app/tests/acceptance/`, mapped 1:1 to every scenario in *User Stories — Final Draft Version 5* (35 user stories, 8 sections). It's separate from the backend lead's existing unit/integration suite (`backend/src/app/tests/*.py`, 151 tests, all passing) — that suite verifies "the code does what it's written to do"; this one verifies "the product does what we promised in the user stories doc."

Each test file corresponds to one user story (`test_us13_submit_reservation.py`, etc.), each test class to one Given/When/Then scenario from the doc, named so a test maps straight back to a scenario number.

Two special markers do double duty as a live gap list:

Marker	Meaning
`@pytest.mark.skip(reason="not implemented: ...")`	The feature described in the scenario doesn't exist in the backend at all yet.
`@pytest.mark.xfail(strict=True, reason="known gap: ...")`	The endpoint exists, but its behavior currently contradicts the doc. `strict=True` means if someone fixes it later without removing the marker, the suite fails loudly instead of staying quietly green.

Running it

```
cd backend && source venv/bin/activate
pytest src/app/tests/acceptance -q      # just acceptance scenarios
pytest -m acceptance -q                  # same, via marker
pytest src/app/tests -q                  # everything: unit suite + acceptance
suite together
```

Coverage status: all 8 sections complete

Section	User Stories	Status
1 — Account & Profile	Admin Invite, US1–7	✅ Done
2 — Tool Listings	US8–11	✅ Done
3 — Browse & Search	US12	✅ Done
4 — Reservations	US13–21	✅ Done
5 — Messaging	US22	✅ Done (all skip — no backend implementation exists)

Section	User Stories	Status
6 — Notifications	US23	✅ Done
7 — Reviews & Ratings	US24–25	✅ Done
8 — Reporting & Moderation	US26–34	✅ Done (mostly skip — see below)

CI/CD integration (GitHub Actions) has not been started — this suite currently only runs locally. That's the natural next step.

Results: full run

299 passed / 49 skipped / 38 xfailed, 0 failures — this is the *combined* count from running `pytest src/app/tests` (151 pre-existing unit tests + 148 new acceptance tests) as a single invocation, confirming the new suite doesn't break anything in the existing one.

Acceptance suite alone: **148 passed / 49 skipped / 38 xfailed** across 235 scenario-tests.

File	User Story	Passed	Skipped	XFailed
`test_us_admin_invite.py`	Admin Invites a New Member	3	1	0
`test_us01_register.py`	1 — Register with Invite Token	4	0	0
`test_us02_verify_email.py`	2 — Verify Email Address	1	0	2
`test_us03_login.py`	3 — Log In Securely	5	0	1
`test_us04_reset_password.py`	4 — Reset Forgotten Password	4	0	1
`test_us05_profile_setup.py`	5 — Set Up Profile	2	2	2
`test_us06_edit_profile.py`	6 — Edit Profile	4	1	2
`test_us07_delete_account.py`	7 — Delete Account	3	1	3
`test_us08_create_listing.py`	8 — Create a Tool Listing	7	2	3
`test_us09_edit_listing_photos.py`	9 — Edit a Tool Listing and Manage Photos	10	3	0
`test_us10_delete_deactivate_listing.py`	10 — Delete or Deactivate a Tool Listing	8	0	1
`test_us11_admin_deactivate_reactivate.py`	11 — Admin Deactivate/Reactivate	4	1	3

File	User Story	Passed	Skipped	XFailed
test_us12_browse_search.py	12 — Browse and Search for Available Tools	6	3	0
test_us13_submit_reservation.py	13 — Submit a Reservation Request	4	0	1
test_us14_approve_deny.py	14 — Approve or Deny Reservation Requests	5	0	0
test_us15_cancel_as_borrower.py	15 — Cancel a Reservation as Borrower	8	0	0
test_us16_cancel_as_owner.py	16 — Cancel a Reservation as Owner	7	0	0
test_us17_confirm_pickup.py	17 — Confirm Tool Pickup	9	1	0
test_us18_auto_cancel_overdue_pickup.py	18 — Auto-Cancel Overdue Pickup	4	1	1
test_us19_timezone_hst_normalization.py	19 — Timezone and Date Normalization	3	2	1
test_us20_confirm_return.py	20 — Confirm Tool Return	10	1	7
test_us21_reservation_history.py	21 — View Reservation History	3	0	1
test_us22_messaging.py	22 — Messaging	0	4	0
test_us23_notifications.py	23 — Receive Notifications	4	0	3
test_us24_leave_review.py	24 — Leave a Rating and Review	16	1	1
test_us25_review_history.py	25 — View a Member's Review History	1	3	0
test_us26_report_listing.py	26 — Member Reports a Listing	0	5	0
test_us27_admin_reviews_reports.py	27 — Admin Reviews Reported Listings	0	4	0
test_us28_admin_manages_categories.py	28 — Admin Manages Tool Categories	0	4	0

File	User Story	Passed	Skipped	XFailed
`test_us29_track_violations.py`	29 — Admin Tracks Member Violations	1	3	0
`test_us30_admin_suspends_member.py`	30 — Admin Suspends a Member Account	4	0	2
`test_us31_admin_reactivates_member.py`	31 — Admin Reactivates a Suspended Member	4	0	1
`test_us32_moderation_history.py`	32 — Admin Views Moderation History	4	0	1
`test_us33_moderation_reports.py`	33 — Admin Generates Moderation Reports	0	4	0
`test_us34_admin_all_reservations.py`	34 — Admin Views All Active Reservations	0	2	1

Findings — gaps between the doc and the current backend

Every gap below is a scenario in the doc the current code doesn't satisfy. None are guesses — each is backed by a failing (xfail) or impossible-to-write (skip) test, and the more surprising ones were spot-checked against the real running server, not just the test DB.

Entire features with no backend implementation

- **Messaging (Section 5 / US22)** — no `Message` model, schema, or router exists at all.
- **Report a listing / admin review of reports (US26–27)** — no `Report` model or endpoints.
- **Admin category management (US28)** — `ToolCategory` is a fixed 5-value Python enum (`HAND_TOOLS`, `POWER_TOOLS`, `GARDEN_TOOLS`, `CLEANING_TOOLS`, `OUTDOOR_GEAR`), not an admin-editable list; the doc's example categories ("Kitchen", "Ladders") don't exist.
- **Member violation tracking (US29)** — `User.violation_count` exists as a column and API field but is never written to anywhere; it's permanently 0.
- **Community moderation reports / CSV export (US33)** — no report-generation endpoint or aggregation logic.
- **Admin reservations overview (US34)** — no platform-wide reservations view; the only reservations endpoint always scopes to the caller's own borrower/owner relationship, even for admins.
- **Public member profiles (US25)** — no `GET /users/{id}` (or similar) endpoint anywhere; there is no way to view another member's profile, reviews, or trust score.
- **`latest_return_time` / lending rules / notes for borrowers** — no such fields exist anywhere on `Tool` (affects US8, US9, US12, and the return-timing scenarios in US20).
- **HST (Hawaii Standard Time) handling** — confirmed via `grep -rniE "hst|hawaii|honolulu|UTC-10"` across the whole backend: zero matches. All date/time logic uses the server's local `date.today()` / UTC timestamps with no timezone-aware conversion anywhere (US19 in full, plus the "evaluated in HST" clauses in US13, US17, US18, US20).
- **Review reminder job (US24 Scenario 9)** — the scheduler has exactly three jobs (auto-cancel pickups, auto-escalate returns, cleanup tokens); no review-reminder job exists.

Implemented, but behavior contradicts the spec

- **1-day rentals are rejected.** `ReservationService.create_reservation` requires `start_date < end_date` (strict), so `start_date == end_date` — the doc's explicit 1-day-rental case (US13 Scenario 5, US19 Scenario 5) — is always rejected.
- **Damage reports flag the wrong user.** `mark_damaged` increments `damage_reported` on `tool.owner_id` (the person filing the report) instead of `reservation.borrower_id` (the person who had the tool). This is a real correctness bug, not just a missing feature — the intended "flag the borrower" behavior is inverted.
- **Damage reports don't affect ratings.** The doc says a damage report should act as a "1-star equivalent" that lowers the borrower's average rating; `ReviewService._recalculate_ratings` only ever averages actual `Review.rating` values and never looks at `damage_reported`.
- **Duplicate damage reports aren't blocked.** `mark_damaged` has no guard against being called twice on the same reservation — a second report silently overwrites the first instead of being rejected.
- **Suspended members can't log in at all.** `AuthService.login` rejects any non-`ACTIVE` user with the generic "Invalid email or password," so a suspended member can't even reach a suspension notice — directly contradicting the doc's "suspended member can still log in."
- **Suspension over-blocks reads.** `get_current_member` requires `ACTIVE` status for *every* member-gated route, including read-only ones (browse tools, view own reservations) — the doc only wants suspended members blocked from write actions, not reads.
- **Admin reactivation has no reason field.** `POST /admin/users/{id}/reactivate` takes no request body; the audit log hardcodes `"Admin reactivation"` regardless of context.
- **Password reset doesn't fully invalidate sessions.** Access tokens are correctly invalidated after a reset (checked against `password_changed_at`), but refresh tokens are not — a stale refresh token can mint a fresh session after a password change.
- **Email verification errors 500 instead of 4xx.** An invalid/expired verification token raises `VerifyTokenError`, which isn't in `app/main.py`'s exception-to-status-code mapping, so it falls through to a bare 500.
- **Logout is a no-op.** Documented as intentional, but it means an access token stays valid after "logout" until natural expiry.
- **No profile field validation.** Display name accepts blank/whitespace-only and 500+ character values on both profile setup and edit.
- **Account deletion has three separate issues:** only checks the caller's own reservations as borrower (an owner with tools out on loan can still delete and strand borrowers); overwrites the display name with the literal string `"Deleted User"` even though the doc requires it be preserved; and is blocked for suspended members even though the doc explicitly allows it.
- **Tool creation** succeeds with zero photos and no description, though the doc requires both.
- **No per-owner duplicate tool-name check.**
- **Deactivating a listing never checks for a `PICKED_UP` reservation** (for both owner self-deactivation and admin deactivation) and never sends notifications to affected borrowers or the owner.
- **The 14-day hard-escalation auto-force-returns overdue items automatically**, contradicting the doc's requirement that a PICKED_UP reservation stay that way until an admin manually resolves it. The 7-day soft escalation also notifies the *borrower*, not the admin as the doc specifies, and sets no admin-visible flag.
- **Notification bodies are missing key details.** New-request, approval/denial, and pickup notifications never include the tool's actual name (just "your tool") or the other party's display name, as the doc requires.
- **`ReservationResponse` exposes only IDs**, not tool name or owner/borrower display names — even though the relationships are already eagerly loaded server-side — so the reservation list/history views

can't show what the doc asks for without extra round-trips.
- `ReviewResponse` **doesn't include the reviewer's display name** either, for the same reason.
- **Admin audit log filtering is partial.** `GET /admin/audit-log` filters by action/target type only — not by admin, date range, or specific listing/member, all of which the doc requires.

Process note

One test initially looked like it caught a bug (a photo delete not reflected on re-fetch) but was actually a **test-harness artifact** — the pytest client fixture shares one DB session/identity map across requests in a way production never does. Verified against the actual running local server with two independent connections before concluding it was fine, then fixed the test instead of reporting a false bug. The same category of issue came up again with IntegrityError-triggered rollbacks (duplicate reservations/reviews) closing the test's ambient transaction — fixed by not issuing further DB-dependent calls after a conflict response in the same test, matching how the existing unit suite already handles it.

Next steps

1. Decide with the team which of the findings above are pre-demo blockers vs. tracked follow-up work — the damage-report-flags-wrong-user and suspended-members-cannot-log-in bugs are probably worth fixing regardless of demo timing.
2. Wire this suite into GitHub Actions (still no CI/CD pipeline exists for this repo).
3. Consider a small Playwright pass for the golden-path UI flow (login → browse → reserve → pickup → return → review) as a complement to this API-level suite.